Project Report: Open Authorization Protocol

02244 Logic for Security

Fahmida Khan (s250116) Zeinab Ali (s243588) Mithra Arul (s257301)

March 16, 2026

Contents

1 Open Authorization Protocol – Zeinab **3**
1.1 Description of the final protocol in AnB . . . 3
1.2 Outline of the design . . . 3
1.2.1 Agents and Initial Knowledge . . . 3
1.2.2 Security Properties . . . 3
1.3 Protocol Design . . . 3
1.4 Formalization in AnB . . . 4

2 Development and Special Tasks (Week 2–4) - Fahmida **4**
2.1 Outline of the design . . . 4
2.2 Description of protocol development and OFMC findings . . . 5
2.2.1 Week 2: Dolev–Yao and static analysis . . . 5
2.2.2 Week 3: Lazy intruder . . . 5
2.2.3 Week 4: Type-flaw resistance and GetPublicKey . . . 5
2.3 AnB formalization . . . 5
2.3.1 Design decisions and development . . . 5
2.3.2 Modeling in AnB . . . 5

3 TLS Channels, Guessable Password, and Final Protocol - Mithra **7**
3.1 Outline of the design . . . 7
3.1.1 Week 5: TLS channels . . . 7
3.1.2 Week 6: Guessable password . . . 7
3.2 Description of any assumptions/simplifications you have made, or remaining problems. . . . 7
3.2.1 Final protocol and assumptions . . . 7
3.3 AnB formalization . . . 7
3.3.1 Summary of special tasks . . . 7
3.3.2 AnB files delivered . . . 7

1 Open Authorization Protocol – Zeinab

1.1 Description of the final protocol in AnB

This section describes the final version of our Open Authorization protocol, including the agents involved, their initial knowledge, and the security goals verified using OFMC.

1.2 Outline of the design

1.2.1 Agents and Initial Knowledge

The protocol involves four agents:

- **A (User)** User A has photos stored at server B and wants to authorize service p to access selected files. A authenticates with the Identity Provider I using the shared password $pw(A, I)$. A initially knows the public key $pk(I)$, the password $pw(A, I)$, and the hash function h.
- **B (Photo Server)** Server B stores the photos belonging to A. B possesses its private key $inv(pk(B))$ and knows $pk(B)$ and $pk(I)$.
- **p (Photo Book Service)** Service p retrieves photos from B in order to create photo books. p possesses $inv(pk(p))$ and knows $pk(p)$ and $pk(I)$.
- **I (Identity Provider)** I authenticates users and issues signed authorization grants. I possesses $inv(pk(I))$ and knows the shared password $pw(A, I)$.

Communication between A and I is modeled as a pseudonymous secure channel (similar to TLS), where the server is authenticated while the client remains pseudonymous.

If A does not initially know $pk(B)$ or $pk(p)$, these can be obtained through a key lookup protocol before executing the authorization protocol.

1.2.2 Security Properties

The protocol aims to establish the following guarantees:

- p authenticates I on the authorization token $fGrant(A, p, B, F, T, Grant)$
- B authenticates I on the authorization token $fGrant(A, p, B, F, T, Grant)$
- I authenticates A on the challenge value NI
- The photo data $fData(Photos)$ remains secret between B and p

In addition, the password $pw(A, I)$ is modeled as a **guessable secret** shared between A and I.

1.3 Protocol Design

The protocol proceeds as follows:

1. Service p sends a request to A asking for permission to access photos stored at B.
2. A contacts the Identity Provider I and forwards the request parameters.
3. I generates a fresh challenge nonce NI and sends it to A.

4. A proves knowledge of the password by sending a hash value containing the password and the session parameters together with the challenge.

5. After verifying the response, I issues a signed authorization grant.

6. p forwards the signed grant to B.

7. B verifies the signature and returns the photos encrypted for p.

The password is never used as an encryption key. Instead, it appears only inside a challenge–response hash that includes fresh session parameters and the nonce NI. This prevents the password from being exposed directly in protocol messages.

The photo data stored at B is modeled by the constant `Photos`.

1.4 Formalization in AnB

```
Protocol: week6_guessablepassword_hardened

Types:
  Agent A, B, I, p;
  Number F, T, R, NI, Grant, Photos;
  Function pk, pw, h, fGrant, fData;

Knowledge:
  A: A, B, I, p, pk(I), pw(A,I), h, fGrant, fData;
  B: A, B, I, p, pk(B), pk(I), pk(p), inv(pk(B)), fGrant, fData;
  p: A, B, I, p, pk(p), pk(I), inv(pk(p)), fGrant, fData;
  I: A, B, I, p, pk(B), pk(p), pk(I), inv(pk(I)), pw(A,I), h, fGrant,
     fData;

Actions:
  p -> A: p, B, F, T, R
  A *->* I: A, p, B, F, T, R
  I *->* A: NI
  A *->* I: A, p, B, F, T, R, NI, h(pw(A,I), A, p, B, F, T, R, NI)
  I -> p: {fGrant(A,p,B,F,T,Grant)}inv(pk(I))
  p -> B: {fGrant(A,p,B,F,T,Grant)}inv(pk(I)), F
  B -> p: {fData(Photos)}pk(p)

Goals:
  pw(A,I) guessable secret between A,I;
  fData(Photos) secret between B,p;
  p authenticates I on fGrant(A,p,B,F,T,Grant);
  B authenticates I on fGrant(A,p,B,F,T,Grant);
  I authenticates A on NI;
```

2 Development and Special Tasks (Week 2–4) - Fahmida

2.1 Outline of the design

The protocol family is developed incrementally from a minimal delegation model toward a structured IDP-based authorization design. The central objective is to allow user A to delegate controlled read access at server B to service P, without sharing long-term credentials. Weeks 2–4 focus on formalizing the threat model, checking executability under the lazy intruder, and improving robustness through typed message formats and explicit key-distribution assumptions.

2.2 Description of protocol development and OFMC findings

2.2.1 Week 2: Dolev–Yao and static analysis

In Week 2 we used a simplified baseline protocol to perform the required static Dolev–Yao analysis under a passive intruder (observe-only, no message injection). In this setting, the intruder learns public and visible fields (agent identities and request context). The intruder does not derive private keys, and cannot decrypt ciphertext encrypted for an uncompromised honest recipient. This baseline analysis gave us a clear starting point: some structures are too weak and must be refined in later weeks to avoid confidentiality/authentication issues in active settings.

2.2.2 Week 3: Lazy intruder

Week 3 introduces identity-provider-issued authorization and the special-task executability check. We verified that role P is executable when P is instantiated by the intruder: the intruder can construct each required outgoing message from previously available knowledge. Operationally, the intruder can receive a token from A, forward it to B, and continue protocol interaction. This is expected for a delegated service role. The intended security condition is not "intruder cannot send," but rather "only correctly issued IDP-bound authorization should be accepted by B."

2.2.3 Week 4: Type-flaw resistance and GetPublicKey

In Week 4 we introduced explicit format constructors (e.g., request/token/data wrappers) to reduce message-type confusion. The core typing claim is: for any two non-variable message patterns, if they unify, they belong to the same intended kind/type. We also added a separate GetPublicKey protocol so that A, initially trusting only $pk(IDP)$, can obtain $pk(X)$ from the identity provider. This removes the unrealistic assumption that A initially knows every party public key.

2.3 AnB formalization

2.3.1 Design decisions and development

The early versions used stronger initial assumptions to get an executable baseline quickly. During development, these assumptions were reduced:

- direct trust assumptions were replaced by IDP-issued authorization,
- typed formats were added to improve message discipline,
- key lookup was split into a dedicated protocol before main delegation flow when needed.

This staged approach made attack traces and modeling limitations visible at each step and allowed targeted refinement.

2.3.2 Modeling in AnB

```
Protocol: OpenAuth_W4_KeyLookup

Types:
  Agent A,I,X;
  Function pk,fKeyReq,fKeyResp;

Knowledge:
  A: A,I,X,pk(A),pk(I),inv(pk(A)),fKeyReq,fKeyResp;
  I: A,I,X,pk(A),pk(I),pk(X),inv(pk(I)),fKeyReq,fKeyResp;
```

```

Actions:
  A -> I: fKeyReq(A,X)
  I -> A: {fKeyResp(X,pk(X))}inv(pk(I))

Goals:
  A authenticates I on fKeyResp(X,pk(X));
```

3 TLS Channels, Guessable Password, and Final Protocol - Mithra

3.1 Outline of the design

3.1.1 Week 5: TLS channels

We model TLS using pseudonymous channel notation. $A \leftrightarrow IDP$ and $P \leftrightarrow B$ use server-authenticated TLS (confidential, authentic). Pseudonymous channels do not ensure freshness; we retain the nonce in the protocol.

3.1.2 Week 6: Guessable password

With $shk(A, IDP)$ marked as guessable in OFMC: the protocol remains secure. The password is used only for authentication, not encryption. The token is signed by IDP's strong key. A guessed password allows impersonation of A to the IDP (inherent to password auth) but does not amplify risk.

3.2 Description of any assumptions/simplifications you have made, or remaining problems.

3.2.1 Final protocol and assumptions

Message flow: $A \to IDP$ (request+MAC); $IDP \to A$ (token); $A \to P$ (token); $P \to B$ (token); $B \to P$ (Photos).

Assumptions: Resource/Scope and Photos are simplified; password modeled as $shk(A, IDP)$; IDP is trusted; A obtains $pk(B)$, $pk(P)$ via GetPublicKey if needed.

3.3 AnB formalization

3.3.1 Summary of special tasks

Week	Special Task
2	Static analysis: intruder cannot forge tokens.
3	Lazy intruder: role P executable.
4	Type-flaw resistant; GetPublicKey protocol.
5	TLS pseudonymous channels; nonce retained.
6	Guessable password: protocol secure.

3.3.2 AnB files delivered

- `OpenAuth_Simple.AnB` — Simplified (Week 2).
- `OpenAuth_Full.AnB` — Full protocol.
- `OpenAuth_Formats.AnB` — Type-flaw resistant.
- `OpenAuth_TLS.AnB` — TLS variant.
- `GetPublicKey.AnB` — Key lookup.

References

- Sebastian Mödersheim, Protocol Security Verification Tutorial. DTU Compute.aa
- OFMC and AnB documentation.
- Course lecture notes, 02244 Logic for Security.